

Differences Between LINQ JOIN and GROUP JOIN Methods

In this article, I will discuss the **Difference Between LINQ Join and Group Join Methods in C# with Examples**. This is one of the frequently asked interview questions. To better understand this, I strongly recommend you read the following two articles, where we explained the LINQ Join and Group Join Methods in Detail.

1. [LINQ Join Method with Real-Time Examples](#)
2. [LINQ Group Join Method with Real-Time Examples](#)

Difference Between LINQ JOIN and GROUP JOIN Method

LINQ provides powerful query capabilities to .NET languages, allowing you to perform complex queries on collections of objects, databases, XML, and more. Among its many methods, JOIN and GROUP JOIN are used to relate elements from two sources of data. Understanding the difference between these two is crucial for effective LINQ querying.

LINQ JOIN Method

The JOIN method in LINQ performs an inner join. It matches elements from two collections based on a key and returns a flat result set that combines elements from both sources. Each matching pair of elements from the two collections is combined into a single result element. The JOIN operation is similar to the INNER JOIN in SQL, meaning it only includes elements that have a match in both collections.

Syntax:

```
var joinQuery = from e1 in collection1
                join e2 in collection2
                on e1.Key equals e2.Key
                select new { e1.Property, e2.Property };
```

Example:

```
var innerJoinQuery = from e in employees
                    join d in departments on e.DepartmentID equals d.ID
                    select new { EmployeeName = e.Name, Department = d.Name };
```

This query would produce a collection where each element consists of an employee's name and their department name, but only for employees who have a matching DepartmentID in the departments collection.

Key Features of LINQ JOIN Method:

- **Basic Functionality:** This operation combines two sequences (collections) based on a key that they share and produces a flat result set. It's similar to the inner join in SQL, where only the matching elements from both sequences are included in the result.
- **Result Structure:** Produces a flat sequence of results. Each result is typically a new type (anonymous or declared) that includes elements from both input sequences.

- **Use Cases:** This method is used when you need to merge data from two sources based on a common key, and you're interested in pairs of matched elements. It's useful for straightforward mappings between two sets of data.

LINQ GROUP JOIN Method

The GROUP JOIN method performs a group join. This method is more complex; it matches elements from two collections based on a key and groups all matching elements from the second collection for each element in the first collection. The result is hierarchical rather than flat, with each element from the first collection paired with a collection of matching elements from the second collection. This is akin to a LEFT OUTER JOIN in SQL, where all elements from the first collection are included, even if they have no matching elements in the second collection.

Syntax:

```
var groupJoinQuery = from e1 in collection1
                      join e2 in collection2
                      on e1.Key equals e2.Key into grouping
                      select new { e1.Key, GroupedItems = grouping };
```

Example:

```
var groupJoinQuery = from d in departments
                      join e in employees on d.ID equals e.DepartmentID into empGroup
                      select new { Department = d.Name, Employees = empGroup };
```

This query would produce a collection of departments, each with a collection of employees that belong to that department. Unlike the Join method, if a department has no employees, the result is an empty collection of employees.

Key Features of LINQ GROUP JOIN Method:

- **Basic Functionality:** This operation is also used to combine two sequences based on a key that they share, but it groups the results from the second sequence for each element in the first sequence. It is more akin to a left outer join in SQL, where all elements from the first sequence are included, and the matching elements from the second sequence are grouped together as a collection within each result.
- **Result Structure:** GROUP JOIN Produces a hierarchical sequence. Each result contains an element from the first sequence and a collection of matching elements from the second sequence. This allows for representing one-to-many relationships within your data.
- **Use Cases:** This is used when you need to join data from two sources and maintain the association between an element from the first source and multiple elements from the second source. It's helpful for scenarios where you want to preserve the one-to-many relationship in the query result.

Key Differences Between LINQ Join and GroupJoin:

- **Result Shape:** JOIN returns a flat result set combining elements from both collections. GROUP JOIN returns a hierarchical result set, pairing each element from the first collection with a collection of matching elements from the second collection.

- **Matching Behavior:** JOIN only includes elements that have matches in both collections (similar to SQL INNER JOIN). GROUP JOIN includes all elements from the first collection, even if there are no matching elements in the second collection (similar to SQL LEFT OUTER JOIN).
- **Use Case:** Use JOIN to combine elements from two collections based on matching keys into a single flat result set. Use GROUP JOIN to maintain the relationships between elements in the first collection and all related elements in the second collection, including retaining elements from the first collection that have no matches.
- **SQL Analogy:** Join is similar to SQL's INNER JOIN, whereas GroupJoin is more akin to LEFT OUTER JOIN, grouping related items together.

Real-Time Example to Understand LINQ Join and Group Join Methods: Schools and Students

Imagine we have two collections: one of the schools and one of the students. Each school has a unique SchoolID, and each student has a StudentID and a SchoolID indicating which school they attend.

Objective: We want to:

- Use JOIN to list all students with their respective schools.
- Use GROUP JOIN to list schools and all their students.

Code Example:

```
using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            // Define the school and student collections
            var schools = new List<School> {
                new School{ SchoolID = 1, Name = "Springfield High" },
                new School{ SchoolID = 2, Name = "Westfield Academy" },
                new School{ SchoolID = 3, Name = "Dot Net School" }
            };
            var students = new List<Student> {
                new Student{ StudentID = 1, SchoolID = 1, Name = "John Doe" },
                new Student{ StudentID = 2, SchoolID = 1, Name = "Jane Smith" },
                new Student{ StudentID = 3, SchoolID = 2, Name = "Will Johnson" },
                new Student{ StudentID = 4, SchoolID = 3, Name = "Sara Taylor" },
                new Student{ StudentID = 5, SchoolID = 3, Name = "Steven Smith" }
            };
            // JOIN example: List all students with their respective schools
            var joinQuery = from s in schools
                           join st in students on s.SchoolID equals st.SchoolID
                           select new { StudentName = st.Name, SchoolName = s.Name };
```

```

        Console.WriteLine("List all students with their respective schools");
        foreach (var studentschool in joinQuery)
        {
            Console.WriteLine($"Student Name: {studentschool.StudentName}
Name: {studentschool.SchoolName}");
        }
        // GROUP JOIN example: List schools along with all their students
        var groupJoinQuery = from s in schools
                              join st in students on s.SchoolID equals st.SchoolID
                              into schoolGroup
                              select new { SchoolName = s.Name, Students = st };

        Console.WriteLine("\nList schools along with all their students");
        foreach (var school in groupJoinQuery)
        {
            Console.WriteLine($"School Name: {school.SchoolName}");
            foreach (var student in school.Students)
            {
                Console.WriteLine($"Student Id: {student.StudentID}, Name: {student.Name}");
            }
        }
        Console.ReadLine();
    }
}

public class School
{
    public int SchoolID { get; set; }
    public string Name { get; set; }
}

public class Student
{
    public int StudentID { get; set; }
    public int SchoolID { get; set; }
    public string Name { get; set; }
}
}

```

Output:

```
List all students with their respective schools
    Student Name: John Doe, School Name: Springfield High
    Student Name: Jane Smith, School Name: Springfield High
    Student Name: Will Johnson, School Name: Westfield Academy
    Student Name: Sara Taylor, School Name: Dot Net School
    Student Name: Steven Smith, School Name: Dot Net School
```

```
List schools along with all their students
School Name: Springfield High
    Student Id: 1, Name: John Doe
    Student Id: 2, Name: Jane Smith
School Name: Westfield Academy
    Student Id: 3, Name: Will Johnson
School Name: Dot Net School
    Student Id: 4, Name: Sara Taylor
    Student Id: 5, Name: Steven Smith
```

In the next article, I will discuss [Parallel LINQ in C#](#) with Examples. In this article, I explain the **Differences Between LINQ Join and Group Join Method in C#** with Examples. I hope you enjoy this article, "Difference Between LINQ Join and Group Join Methods in C#."